KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK
Faculty of Applied Science / Faculty of Information Technology
FINAL EXAMINATION — SEMESTER 1, ACADEMIC YEAR 2026 (DETAILED VERSION 3)

Course: 060243106 & 060223119 Data Structures and Algorithms | Instructor: Assoc. Prof. Dr. Pradit Pitaksatheinkul
Duration: 3 Hours (180 Mins) | Total Score: 70 Raw Marks (35% Final Grade) | Open Book & Scientific Calculator Allowed

Student Name: ..... Student ID: ..... Section: .....

Question 1: Hashing & Open Addressing with Double Hashing (10 Marks)

Given an empty Hash Table of size TableSize = 19 (indices 0 to 18) using Open Addressing with Double Hashing.
- Primary Hash Function: $h_0(X) = X \bmod 19$
- Collision Resolution Function: $f(i) = i \times hash_2(X) = i \times (17 - (X \bmod 17))$ for probe $i = 0, 1, 2, ...$
- Probe Sequence Address: $h_i(X) = (h_0(X) + f(i)) \bmod 19$

Keys to insert sequentially: [26, 45, 70, 85, 102, 123, 143, 155]

1.1 Step-by-Step Probe Calculation & Collision Tracing: [3 Marks]

Show the complete hash calculations for each key. For keys experiencing collisions, record every probe step:

Key (X)	$h_0(X) = X \bmod 19$	$hash_2(X) = 17 - (X \bmod 17)$	Probe Sequence & Collision Trace	Final Slot	Collisions
26	.....	.....	.....	.....	.....
45	.....	.....	.....	.....	.....
70	.....	.....	.....	.....	.....
85	.....	.....	.....	.....	.....
102	.....	.....	.....	.....	.....
123	.....	.....	.....	.....	.....
143	.....	.....	.....	.....	.....
155	.....	.....	.....	.....	.....

1.2 Final Hash Table Array Representation (Element & Info): [2 Marks]

Fill in the table array below. Record Element (Key) and Info ('A' = Active, '-' = Empty):

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Element																			
Info																			

1.3 Collisions Breakdown & Max Colliding Keys: [2 Marks]

- Keys with 0 collisions: ..... | Keys with 1 collision: .....
- Which key(s) experienced the maximum number of collisions? .....
  Number of collisions: ..... collisions each | Slot locations of collisions: .....
- Total cumulative number of collisions across all 8 insertions: ..... collisions.

1.4 Load Factor & Rehashing Threshold Analysis: [3 Marks]

- Current Load Factor ($\lambda = N / TableSize$): ..... (State exact fraction and percentage)
- If Rehashing is triggered when the load factor reaches $\lambda \geq 80\%$:
  (a) Maximum number of elements allowed before Rehashing: ..... elements.
  (b) How many MORE keys must be inserted before Rehashing occurs? ..... more keys.
  (c) When Rehashing occurs, what must the New Table Size be? State the prime number rule and exact size: .....

Question 2: Priority Queue (Binary Min-Heap) Tracing & Operations (10 Marks)

Given an empty Binary Min-Heap represented as a 1D array using 1-based indexing (Index 0 is unused / sentinel).
Keys to insert sequentially: [64, 14, 24, 1, 20, 2, 5, 10, 6, 8, 15, 20] (Total 12 keys).

2.1 Fundamental Properties & 1D Array Mapping: [1 Mark]

• State the 2 fundamental properties of a Binary Min-Heap:
1. Structure Property: .....
2. Heap-Order Property: .....
• 1-Based Array Formulas for index i: Left Child = ... | Right Child = ... | Parent = ... (Why is index 0 unused? .....)

2.2 Four Vertical Array Tracing Tables (Past Exam Official Format): [4 Marks]

Fill in the array contents after each operation. Write only final states. Follow classroom program behavior:

Index	2.1 After Insert 12 Keys	2.2 After deleteMin() #1	2.3 After Insert Key 3	2.4 After deleteMin() #2
[0] (Unused)	-	-	-	-
[1]				
[2]				
[3]				
[4]				
[5]				
[6]				
[7]				
[8]				
[9]				
[10]				
[11]				
[12]		N/A		N/A

2.3 Tree Structure Diagrams: [2 Marks]

Draw the complete binary tree structure corresponding to Table 2.1 (All 12 keys) and Table 2.4 (Final state):

Tree Diagram: 2.1 After 12 Inserts	Tree Diagram: 2.4 Final State (After deleteMin #2)

2.4 'hole' Variable Tracing & Algorithmic Steps: [2 Marks]

• deleteMin() #1: Root (1) removed. Last element temp = 20 placed at hole. Trace hole positions during percolateDown:
hole starts at index: ..... → compares children at ..... → hole moves to: ..... → compares children at ..... → hole moves to: .....
• Insert Key 3: hole starts at index 12. Trace hole positions during percolateUp:
hole at index 12 compares parent at index ..... → hole moves to: ..... → compares parent at index ..... → hole moves to: .....

2.5 Time Complexity Analysis: [1 Mark]

• Worst-case time complexity of insert: ..... | Average-case time complexity of insert: .....
• Worst-case time complexity of deleteMin: ..... (Explain briefly why average insert is O(1):
.....)

Question 3: Insertion Sort Tracing & Position Moves (10 Marks)

3.1 Worst-Case Mathematical Analysis (N = 12 Elements): [3 Marks]

• Write down the initial array of 12 elements representing the Worst-Case input (Warning: Wrong initial input = 0 marks):
Initial Array: [.....]
• State the formula for total position moves (shifts): ..... | Total Position Moves for N = 12: = ... moves.
• Total element comparisons in Worst Case: = ... comparisons.

3.2 Array Tracing Table: Given input [61, 52, 12, 24, 8, 35, 32, 5, 71, 68, 18, 2]: [4 Marks]

Pass (p)	Key	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	Moves
Initial	-	61	52	12	24	8	35	32	5	71	68	18	2	-
p = 1	52													
p = 2	12													
p = 3	24													
p = 4	8													
p = 5	35													
p = 6	32													
p = 7	5													
p = 8	71													
p = 9	68													
p = 10	18													
p = 11	2													

3.3 Specific Pass Verification & Inversions Sum: [3 Marks]

• Pass p = 3 (Key 24): Comparisons made: ..... | Position Moves in this pass: ..... moves.
• Pass p = 6 (Key 32): Comparisons made: ..... | Position Moves in this pass: ..... moves.
• Grand Total of Position Moves from start to finish: = ... moves | Total Inversions in original array: = ... inversions.

Question 4: Selection Sort & Bubble Sort Tracing (10 Marks)

Given the 8-element unsorted array: [45, 18, 72, 31, 12, 88, 25, 60]

4.1 Selection Sort Tracing (Pass-by-Pass): [5 Marks]

For each round i (0 to 6), find the minimum element in unsorted subarray and swap with a[i]:
• Round 0: min_idx = ... (val = ...) → Swap with index 0 → Array: [.....]
• Round 1: min_idx = ... (val = ...) → Swap with index 1 → Array: [.....]
• Round 2: min_idx = ... (val = ...) → Swap with index 2 → Array: [.....]
• Round 3: min_idx = ... (val = ...) → Swap with index 3 → Array: [.....]
• Round 4: min_idx = ... (val = ...) → Swap with index 4 → Array: [.....]
• Total passes = ... | Total element comparisons = ... | Total swaps = ... swaps.

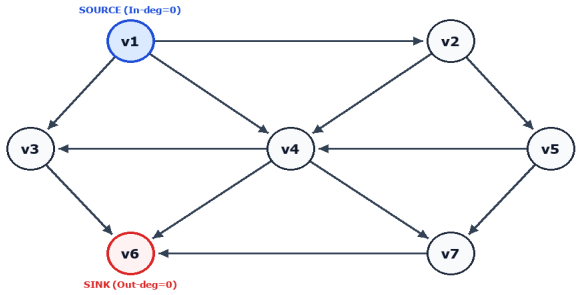
4.2 Bubble Sort Tracing & Early Termination: [5 Marks]

• Pass 1 Tracing: Detail all 7 adjacent comparisons and swaps:
(45, 18) → ..... | (45, 72) → ..... | (72, 31) → ..... | (72, 12) → ..... | (72, 88) → ..... | (88, 25) → ..... | (88, 60) → .....
Array after Pass 1: [.....] | Swaps in Pass 1: ..... swaps.
• Total swaps from start to finish: = ... swaps | Total outer loop passes made: ..... passes.
• If optimized with a swap flag, at which pass does the algorithm terminate early? Pass: ..... (Explain: .....)

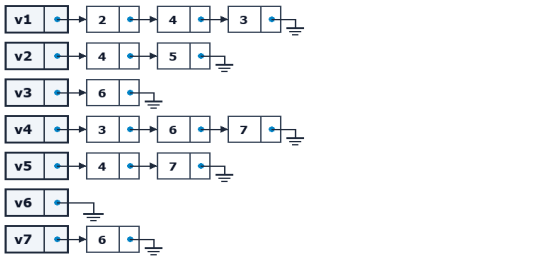
Question 5: Topological Sort (Kahn's Algorithm on 7-Vertex DAG) (10 Marks)

A Directed Acyclic Graph (DAG) with 7 vertices (1 to 7) and 12 directed edges is defined by the diagram and Adjacency List below.
We execute Kahn's Algorithm using an Indegree table and a Circular Queue.

Directed Acyclic Graph (DAG) [Lecture 10 & Exam 2026 Q5]: |V|=7, |E|=12



Adjacency List: Directed Acyclic Graph (|V|=7, |E|=12)



5.1 Indegree Array Processing Table across All 7 Rounds: [4 Marks]

Record the Indegree count for each vertex in each round. Cross out values as vertices are removed:

Round	Vertex Dequeued	v_1	v_2	v_3	v_4	v_5	v_6	v_7
Initial	-							
Round 1								
Round 2								
Round 3								
Round 4								
Round 5								
Round 6								
Round 7								

5.2 Circular Queue Tracing (Capacity 7, Slots 0 to 6): [2 Marks]

The queue uses circular array indexing: enqueue: back = (back+1)%7 and dequeue: front = (front+1)%7.

- Complete sequence of vertices Enqueued into Queue: .....
- Complete sequence of vertices Dequeued from Queue: .....

Slot Index	[0]	[1]	[2]	[3]	[4]	[5]	[6]
Final Queue Array							

- Final Pointer Locations: front is at index: ..... | back is at index: ..... | currentSize = ...
- Final Queue Status: [ ] Empty (0 elements) [ ] Non-empty (Data remains)

5.3 Final Topological Ordering: [3 Marks]

Write the resulting Topological Ordering sequence produced by Kahn's algorithm:
Answer: .....
* CRITICAL ZERO-MARK WARNING: Must write comma-separated sequence (e.g. 5, 7, 2, 1, 3, 4, 6). DO NOT write arrows (5 → 7 → 2...) which receives 0 marks!

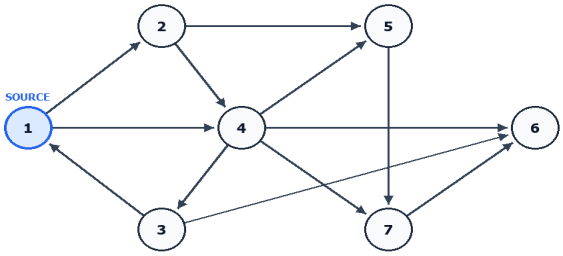
5.4 Cycle Detection & DAG Verification: [1 Mark]

How does Kahn's algorithm detect if a graph contains a cycle (is NOT a DAG)? State the exact mathematical condition:
Answer: .....

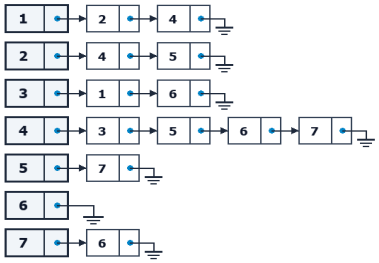
Question 6: Unweighted Shortest Path Algorithm (BFS) (10 Marks)

A directed unweighted graph with 7 vertices (1 to 7) and 12 directed edges is given below. (Source Vertex = 1).
We execute the BFS-based unweighted shortest path algorithm using a processing table and a Queue of capacity 7.

Directed Graph G = (V, E) [Exam 2026 Question 6]: Source Vertex = 1



Adjacency List: Directed Graph (Source = 1, 7 Vertices)



6.1 BFS Execution Table for Shortest Path (Source Vertex = 1): [4 Marks]

Fill in the processing array table containing Known (T / F), Distance (d_v), Previous Path (p_v), and Shortest Route:

Vertex (v)	Known (T / F)	Distance (d_v)	Previous Path (p_v)	Shortest Route from Source (1)
v_1				
v_2				
v_3				
v_4				
v_5				
v_6				
v_7				

6.2 Circular Queue State upon Completion (Capacity 7, Slots 0 to 6): [2 Marks]

• Enqueue Sequence: ..... | Dequeue Sequence: .....

Slot Index	[0]	[1]	[2]	[3]	[4]	[5]	[6]
Final Queue Array							

• front is at index: ..... | back is at index: ..... | currentSize: ..... | Status: [ ] Empty [ ] Non-empty

6.3 Shortest Path Reconstruction & Distance (4 Destinations): [3 Marks]

* CRITICAL PROFESSOR'S RULE: Must write comma-separated sequences ONLY (e.g. 1, 4, 5). DO NOT write arrows (1 → 4 → 5). Distance is measured in number of EDGES!

• Shortest Path to Vertex 5: Distance = ... edges | Route Sequence: = ...
• Shortest Path to Vertex 7: Distance = ... edges | Route Sequence: = ...
• Shortest Path to Vertex 6: Distance = ... edges | Route Sequence: = ...
• Shortest Path to Vertex 3: Distance = ... edges | Route Sequence: = ...

6.4 Algorithm Complexity & BFS Property: [1 Mark]

• Time complexity of unweighted shortest path with Adjacency List: .....
• Why does BFS guarantee the shortest path in unweighted graphs but fail in weighted graphs?
.....

Question 7: Advanced Theory, Structural Properties & Calculations (10 Marks)

Answer the following questions concisely. All calculations must be computed as exact integers or percentages (No formulas/powers).

7.1 Structural Properties (Data Structures Classification): [2 Marks]

(a) Which fundamental data structure possesses exactly two formal properties? State the data structure and define both properties:
- Data Structure: .....
- Property 1: .....
- Property 2: .....

(b) Which tree data structure possesses exactly three formal properties? State the data structure and list all three properties:
- Data Structure: .....
- Properties: .....

7.2 Hash Function Theoretical Properties & Realistic Limitation: [2 Marks]

- State the 3 ideal properties of a theoretical hash function:
  1. .....
  2. .....
  3. .....
- Which of these 3 properties is impossible to achieve in practice? .....
- State the two fundamental reasons why it is impossible:
  Reason 1: .....
  Reason 2: .....

7.3 Graph Memory Waste Percentage (Sparse Graph Analysis): [2 Marks]

A directed graph has |V| = 7 vertices and |E| = 12 edges.
- Total cells in Adjacency Matrix: ..... cells.
- Number and percentage of cells containing 1 (active edges): ..... cells (..... %)
- Number and percentage of cells containing 0 (wasted space): ..... cells (..... %)
- Total memory cells required if represented as an Adjacency List (|V| + |E|): ..... cells.
- Percentage of memory saved by Adjacency List compared to Adjacency Matrix: ..... %

7.4 Complete Graph Edge Calculations: [1 Mark]

(a) A complete undirected graph K_{16} has |V| = 16 vertices. Calculate the total number of edges as an exact integer:
Calculation & Answer: ..... edges.

(b) A complete directed graph with 16 vertices without self-loops has: ..... directed edges.

7.5 Complete Binary Tree Node Bounds: [2 Marks]

For a Complete Binary Tree where the Root is at height h = 0, calculate exact integer bounds:
- Height h = 10: Minimum nodes = ... nodes | Maximum nodes = ... nodes.
- Height h = 15: Minimum nodes = ... nodes | Maximum nodes = ... nodes.
- Height h = 16: Minimum nodes = ... nodes | Maximum nodes = ... nodes.

(Strict rule: Answers MUST be computed exact integers, NOT exponential expressions like 2^{10} or 2^{16})

7.6 Binary Heap 1D Array Formulas: [1 Mark]

For a node located at index i in a 1-based array implementation of a Binary Heap, state the exact formula for:
- Left Child Index: ..... | Right Child Index: ..... | Parent Index: .....
- Why must integer floor division (i // 2) be used instead of rounding up? .....

***** END OF EXAMINATION PAPER — WISHING YOU THE BEST OF LUCK! *****